ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH

Đề bài tập lớn cho Nhóm 4

Bài tập lớn số 1

GVHD: TS. Lê Thành Sách
SV thực hiện: Nguyễn Hữu Trưởng - 2470573
Nguyễn Nhật Thanh - 2570316
Võ Thị Bích Phượng - 2470570

Tp. Hồ Chí Minh, Tháng 03/2026

Mục lục

1 Giới thiệu đề tài **3**
1.1 Ngữ cảnh và Mục tiêu . . . 3
1.2 Phương pháp và Mô hình thực nghiệm . . . 3
1.3 Tập dữ liệu sử dụng . . . 3
1.3.1 Tập dữ liệu ảnh: CIFAR-10 . . . 3
1.3.2 Tập dữ liệu văn bản: IMDB Movie Reviews . . . 4
1.3.3 Tập dữ liệu đa phương thức: COCO Captions 2017 . . . 4
1.4 Metric đánh giá (Evaluation Metrics) . . . 4

2 Giới thiệu về thuật toán CNN **4**
2.1 Động lực và ý tưởng cốt lõi . . . 5
2.2 Các thành phần toán học và kiến trúc . . . 5
2.2.1 Phép toán tích chập (Convolution Operation) . . . 5
2.2.2 Hàm kích hoạt phi tuyến (Non-linearity) . . . 6
2.2.3 Pooling và Bất biến dịch chuyển . . . 6
2.3 Mô hình hóa chuỗi (Sequence Modeling) với RNN . . . 7
2.4 Hạn chế của RNN truyền thống và giải pháp LSTM . . . 7

3 Phân loại văn bản với RNN & Transformer **8**
3.1 Tổng quan về Xử lý ngôn ngữ tự nhiên và Phân tích cảm xúc . . . 8
3.2 Cơ sở lý thuyết . . . 8
3.2.1 Tiền xử lý dữ liệu và Không gian nhúng từ (Word Embeddings) . . . 8
3.2.1.1 Tiền xử lý và Mã hóa (Tokenization) . . . 8
3.2.1.2 Biểu diễn từ phân tán với GloVe (Global Vectors for Word Representation) . . . 8
3.2.2 Mạng Nơ-ron Hồi quy tiến hóa: Từ RNN đến Bi-LSTM . . . 9
3.2.2.1 Mạng LSTM (Long Short-Term Memory) . . . 9
3.2.2.2 Mạng LSTM hai chiều (Bidirectional LSTM) và Cơ chế Gộp (Pooling) 9
3.2.3 Kiến trúc Transformer và Mô hình BERT . . . 10
3.2.3.1 Cơ chế Tự chú ý (Self-Attention) . . . 10
3.2.3.2 Mô hình BERT . . . 10
3.2.4 Hàm mất mát và Kỹ thuật Tối ưu hóa . . . 10
3.2.4.1 Hàm mất mát (Loss Functions) . . . 10
3.2.4.2 Kỹ thuật chống Quá khớp (Overfitting) . . . 10
3.2.5 Các độ đo đánh giá (Evaluation Metrics) . . . 11
3.3 Mô hình đề xuất . . . 11
3.3.1 Phân tích Khám phá và Phân chia Dữ liệu (EDA & Data Splitting) . . . 11
3.3.2 Tiền xử lý Văn bản và Biểu diễn Dữ liệu . . . 12
3.3.2.1 Biểu diễn cho mô hình Bi-LSTM . . . 12
3.3.2.2 Biểu diễn cho mô hình BERT . . . 13
3.3.3 Kiến trúc Đề xuất 1: Mạng Bi-LSTM tích hợp Gộp đặc trưng (Pooling) . . . 13
3.3.4 Kiến trúc Đề xuất 2: Tinh chỉnh Mô hình BERT (Fine-tuning) . . . 13
3.3.5 Cấu hình Huấn luyện và Chiến lược Tối ưu hóa . . . 13
3.3.5.1 Tối ưu hóa Mô hình Bi-LSTM . . . 13
3.3.5.2 Tối ưu hóa Mô hình BERT . . . 13
3.3.5.3 Tóm tắt siêu tham số thực nghiệm . . . 14
3.4 Kết quả Thực nghiệm và Đánh giá . . . 14
3.4.1 Môi trường Thực nghiệm và Cấu hình Siêu tham số . . . 14
3.4.2 Phân tích Động lực học Quá trình Huấn luyện . . . 14
3.4.2.1 Động lực học của Mô hình BERT . . . 14
3.4.2.2 Động lực học của Mô hình Bi-LSTM . . . 15
3.4.3 Đánh giá Định lượng trên Tập Kiểm thử (Test Set) . . . 15
3.4.4 Phân tích Ma trận Nhầm lẫn (Confusion Matrix) . . . 16
3.4.5 Đánh giá Định tính: Thử nghiệm Suy diễn (Inference Case Studies) . . . 17
3.4.6 Tiểu kết . . . 17

4 Phân loại Đa phương thức với CLIP **17**
4.1 Giới thiệu . . . 17
4.2 Tập dữ liệu COCO 2017 . . . 18
4.2.1 Giới thiệu về COCO . . . 18
4.2.2 Xử lý và Lọc Dữ liệu . . . 18
4.3 Kiến trúc CLIP . . . 19
4.3.1 Tổng quan về CLIP . . . 19
4.3.2 Các thành phần chính . . . 19
4.3.3 Contrastive Learning . . . 20
4.3.4 Tại sao CLIP phù hợp cho Multimodal Classification? . . . 20
4.4 Ba Phương pháp So sánh . . . 20
4.4.1 Zero-Shot Classification . . . 20
4.4.2 Few-Shot Classification (Frozen Backbone) . . . 21
4.4.3 Full Fine-Tuning (Unfrozen Backbone) . . . 22
4.5 Thiết lập Thực nghiệm . . . 23
4.5.1 Môi trường và Phần cứng . . . 23
4.5.2 Chia tách dữ liệu . . . 23
4.5.3 Metrics đánh giá . . . 23
4.6 Kết quả và Phân tích . . . 24
4.6.1 So sánh Tổng quan . . . 24
4.6.2 Thời gian Huấn luyện và Inference . . . 25
4.6.3 Phân tích theo K (Samples per Class) . . . 25
4.6.4 Confusion Matrices . . . 25
4.6.5 Per-class Performance . . . 26
4.7 Kết luận . . . 27
4.7.1 Tổng kết Kết quả . . . 27
4.7.2 Trade-offs giữa các Phương pháp . . . 27
4.7.3 Khuyến nghị Sử dụng . . . 27
4.7.4 Hướng Phát triển . . . 28

Tài liệu tham khảo **28**

1 Giới thiệu đề tài

1.1 Ngữ cảnh và Mục tiêu

Trong khuôn khổ môn học Học sâu và ứng dụng trong thị giác máy tính, nhóm chúng em thực hiện Bài tập lớn số 1 với chủ đề tập trung vào bài toán phân loại (classification). Mục tiêu cốt lõi của đề tài là vận dụng và đánh giá hiệu năng của các kiến trúc mạng nơ-ron học sâu tiên tiến trên ba loại dữ liệu khác biệt: ảnh, văn bản và đa phương thức (ảnh kết hợp văn bản).

Thông qua quá trình triển khai, nhóm hướng tới việc đạt được các mục tiêu cụ thể sau:

- Chuẩn bị dữ liệu và thiết lập quy trình huấn luyện, đánh giá chuẩn xác.
- Vận dụng các mô hình pretrained đa dạng để giải quyết bài toán phân loại.
- So sánh, phân tích kết quả giữa các họ mô hình học sâu khác nhau thông qua bảng số liệu và biểu đồ trực quan.

1.2 Phương pháp và Mô hình thực nghiệm

Để thực hiện so sánh một cách toàn diện theo yêu cầu của Bài tập lớn, nhóm tiến hành xây dựng và huấn luyện (fine-tune) các mô hình sau:

- **Với dữ liệu ảnh:** Đối chiếu hiệu năng giữa mạng tích chập truyền thống CNN (Convolutional Neural Networks) và kiến trúc ViT (Vision Transformer).
- **Với dữ liệu văn bản:** Đánh giá sự khác biệt giữa mạng nơ-ron hồi quy RNN (cụ thể là cấu trúc LSTM) và kiến trúc Transformer.
- **Với dữ liệu đa phương thức:** Khảo sát khả năng phân loại thông qua hai chiến lược tiếp cận là Zero-shot classification và Few-shot classification.

1.3 Tập dữ liệu sử dụng

Nhằm đảm bảo các mô hình được đánh giá một cách thuyết phục nhất, nhóm đã lựa chọn ba tập dữ liệu đáp ứng tiêu chí về độ phân biệt lớp và số lượng mẫu huấn luyện từ vài nghìn mẫu trở lên:

- **Tập dữ liệu ảnh - CIFAR-10:** Đây là một trong những tập dữ liệu tiêu chuẩn và được gợi ý sử dụng cho bài toán phân loại ảnh.
- **Tập dữ liệu văn bản - IMDB:** Được lựa chọn cho tác vụ phân loại văn bản (text classification).
- **Tập dữ liệu đa phương thức - COCO-caption 2017:** Đây là tập dữ liệu cung cấp các cặp ảnh và văn bản thực sự, cùng mô tả một thực thể hoặc sự kiện, đáp ứng đúng yêu cầu của bài toán phân loại đa phương thức.

1.3.1 Tập dữ liệu ảnh: CIFAR-10

Tập dữ liệu CIFAR-10 bao gồm 60.000 ảnh màu có kích thước 32x32 pixel, được phân bổ đều vào 10 lớp vật thể và động vật khác nhau (như máy bay, ô tô, chó, mèo,...).

- **Đáp ứng tiêu chí:** Tập dữ liệu có 50.000 ảnh dùng để huấn luyện, thỏa mãn hoàn toàn yêu cầu có từ vài nghìn mẫu trở lên (lớn hơn 5.000 mẫu). Đồng thời, với 10 lớp phân biệt, CIFAR-10 vượt mức tối thiểu 5 lớp mà đề bài bắt buộc.
- **Lý do lựa chọn:** Đây là một tập dữ liệu có độ khó vừa phải, các lớp có sự đa dạng về góc chụp và bối cảnh, khắc phục được sự đơn giản quá mức của tập MNIST. Điều này tạo ra một môi trường thử nghiệm hoàn hảo để nhóm đánh giá và so sánh năng lực trích xuất đặc trưng không gian của mạng tích chập (CNN) so với khả năng nắm bắt ngữ cảnh toàn cục của kiến trúc Vision Transformer (ViT).

1.3.2 Tập dữ liệu văn bản: IMDB Movie Reviews

Tập dữ liệu IMDB chứa 50.000 văn bản đánh giá phim tiếng Anh, với độ dài và cấu trúc ngữ nghĩa phong phú, đa dạng. Kích thước tập dữ liệu này hoàn toàn đáp ứng điều kiện về số lượng mẫu huấn luyện.

- **Cách đáp ứng ràng buộc 5 lớp:** Thông thường, IMDB được dùng cho bài toán phân loại nhị phân (Tích cực/Tiêu cực). Tuy nhiên, vì đề bài quy định rõ bài toán nhị phân là không hợp lệ để so sánh mô hình và phải có ít nhất 5 lớp, nhóm đã tiến hành tiền xử lý (preprocessing) lại nhãn của tập dữ liệu. Dựa trên điểm số rating gốc của người dùng (từ 1 đến 10 sao), nhóm đã phân loại lại tập dữ liệu thành 5 lớp cảm xúc tương ứng (ví dụ: Rất tiêu cực, Tiêu cực, Trung lập, Tích cực, Rất tích cực).
- **Lý do lựa chọn:** Các đoạn đánh giá trên IMDB thường có cấu trúc câu dài và phức tạp. Việc phân loại thành 5 mức độ cảm xúc yêu cầu mô hình phải hiểu được các từ khóa nhấn mạnh và ngữ cảnh đảo ngược. Nhờ đó, nhóm có thể thấy rõ sự chênh lệch về hiệu năng khi xử lý chuỗi dài giữa mạng nơ-ron hồi quy RNN (LSTM) và cơ chế tự chú ý của Transformer.

1.3.3 Tập dữ liệu đa phương thức: COCO Captions 2017

COCO (Common Objects in Context) Captions là bộ dữ liệu tiêu chuẩn quy mô lớn dùng để nghiên cứu mối liên hệ giữa hình ảnh và ngôn ngữ tự nhiên.

- **Đáp ứng tiêu chí:** Tập dữ liệu cung cấp các cặp ảnh - văn bản thực sự. Mỗi bức ảnh đi kèm với khoảng 5 câu mô tả (captions) do con người trực tiếp chú thích, đảm bảo sự liên kết chặt chẽ về mặt thực thể và sự kiện, tuân thủ tuyệt đối quy định không được phép ghép ảnh và văn bản một cách ngẫu nhiên. Đây cũng là một trong những tập dữ liệu được gợi ý trực tiếp trong hướng dẫn.
- **Lý do lựa chọn:** Sự đa dạng và độ phức tạp cao của COCO Captions cung cấp một nền tảng thử nghiệm lý tưởng để nhóm đánh giá sức mạnh của các mô hình đa phương thức. Cụ thể, nhóm sử dụng bộ dữ liệu này để đối chiếu hiệu quả của hai phương pháp tiếp cận: Zero-shot classification (khả năng mô hình nhận diện các lớp chưa từng thấy trong quá trình huấn luyện) và Few-shot classification (khả năng học nhanh từ một lượng rất nhỏ dữ liệu mẫu).

1.4 Metric đánh giá (Evaluation Metrics)

Để đảm bảo sự công bằng và khách quan khi đối chiếu kết quả giữa các kiến trúc mô hình khác nhau (CNN vs. ViT, RNN vs. Transformer), nhóm thống nhất sử dụng chung một hệ thống các độ đo (metrics).

- **Accuracy (Độ chính xác toàn cục):** Đây là metric bắt buộc được sử dụng để đánh giá tổng quan tỷ lệ dự đoán đúng trên toàn bộ tập dữ liệu.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

- **F1-Score:** Trong các trường hợp các lớp dữ liệu có sự phân bổ không đồng đều (mất cân bằng lớp), F1-Score được bổ sung để đánh giá mô hình một cách toàn diện hơn, tránh sai lệch do độ chính xác ảo. F1-Score là trung bình điều hòa của Precision (Độ xác thực) và Recall (Độ bao phủ):

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2}$$

- **Ma trận nhầm lẫn (Confusion Matrix):** Nhóm bổ sung thêm ma trận này nhằm trực quan hóa các trường hợp mô hình nhận diện sai, từ đó phục vụ cho việc phân tích lỗi sâu hơn ở phần sau.

2 Giới thiệu về thuật toán CNN

Trong bài toán OCR, Mạng Nơ-ron Tích chập (Convolutional Neural Network - CNN) không đơn thuần là một bộ phân loại (classifier) mà đóng vai trò là bộ mã hóa đặc trưng thị giác (*Visual Feature Encoder*). Mục tiêu của nó là ánh xạ hình ảnh đầu vào $I \in \mathbb{R}^{H \times W \times C}$ (với H, W, C lần lượt là chiều cao, chiều rộng và số kênh màu) thành một bản đồ đặc trưng (feature map) trừu tượng $X \in \mathbb{R}^{H' \times W' \times D}$.

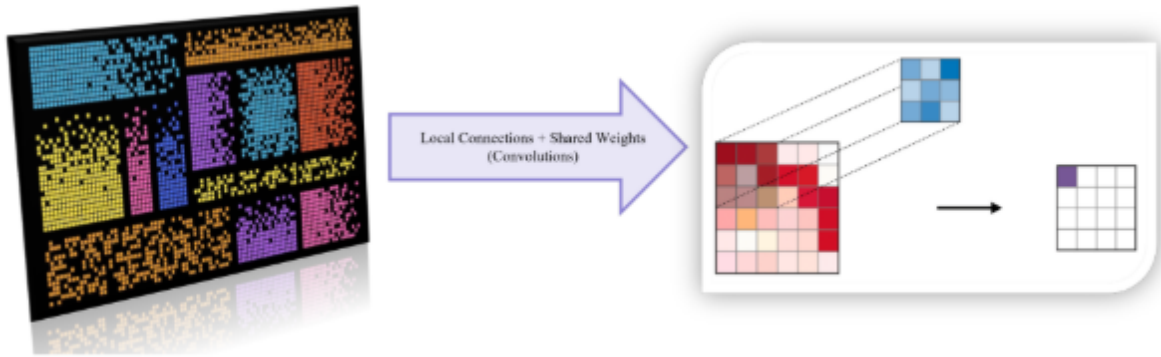
2.1 Động lực và ý tưởng cốt lõi

Trước khi đi sâu vào kiến trúc, cần xem xét lý do CNN vượt trội hơn các mạng MLP (Multi-layer Perceptron) truyền thống trong xử lý ảnh:

1. **Vấn đề bùng nổ tham số:** Nếu áp dụng MLP fully-connected trực tiếp lên một ảnh có kích thước lớn (ví dụ 1000×1000 pixels), số lượng trọng số sẽ tăng lên khổng lồ (lên tới 10^{12} cho một lớp ẩn), dẫn đến việc mô hình dễ bị *overfitting* và tiêu tốn tài nguyên tính toán không cần thiết.

2. **Cấu trúc không gian (Spatial Structure):** MLP coi các pixel là độc lập, trong khi hình ảnh có tính chất cục bộ (*feature localization*) - các pixel lân cận có mối quan hệ mật thiết, và tính chất bất biến vị trí (*translation invariance*) - một đặc trưng (như cạnh, góc) có ý nghĩa như nhau dù xuất hiện ở vị trí nào trong ảnh.

Để giải quyết vấn đề này, CNN sử dụng ba cơ chế tính toán chính: **Local Receptive Fields** (Vùng cảm nhận cục bộ), **Shared Weights** (Chia sẻ trọng số) và **Spatial Subsampling** (Lấy mẫu không gian).



Hình 1: *Cơ chế tính toán của CNN*

2.2 Các thành phần toán học và kiến trúc

2.2.1 Phép toán tích chập (Convolution Operation)

Đây là lớp cốt lõi dùng để trích xuất các đặc trưng cục bộ. Thay vì kết nối đầy đủ, CNN sử dụng một bộ lọc (kernel/filter) K có kích thước nhỏ $K_h \times K_w$ trượt qua toàn bộ ảnh.

Với một ảnh đầu vào I và bộ lọc K, giá trị của bản đồ đặc trưng đầu ra S tại vị trí (i, j) được tính bằng công thức tích chập rời rạc:

$$S(i,j) = (I * K)(i,j) = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} I(i-u, j-v) \cdot K(u,v) \tag{3}$$

Ngoài ra, trong các ứng dụng thực tế như OCR, một lớp tích chập thường được biểu diễn dưới dạng:

$$Y = \sigma(W * X + b) \tag{4}$$

trong đó W là kernel, X là input, b là bias, và σ là hàm activation.

Ngoài kích thước bộ lọc, hai siêu tham số quan trọng kiểm soát kiến trúc là:

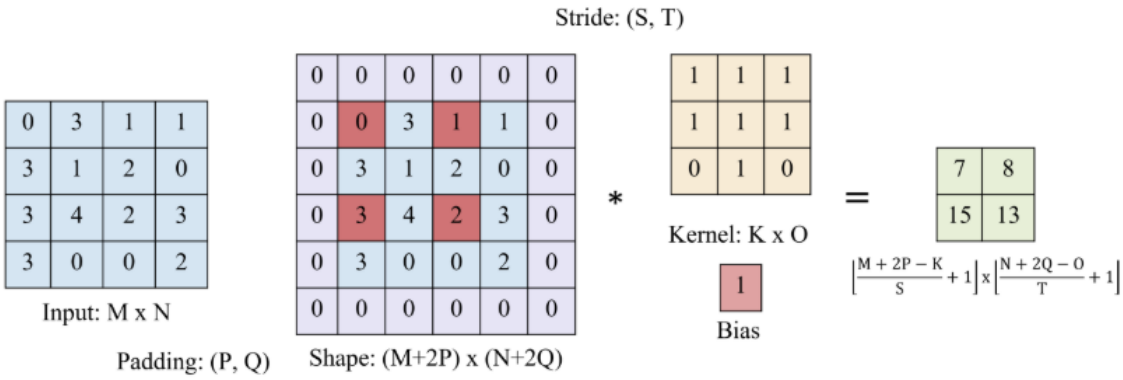
- **Padding (P):** Thêm các pixel giả (thường là 0) xung quanh biên ảnh để kiểm soát kích thước đầu ra và tránh mất thông tin ở rìa ảnh.

- **Stride (S):** Bước nhảy của bộ lọc khi trượt trên ảnh. Stride càng lớn thì kích thước đầu ra càng nhỏ (giảm độ phân giải không gian).

Kích thước của Feature Map đầu ra ($H_{out} \times W_{out}$) được tính theo công thức:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K_h}{S} \right\rfloor + 1; \quad W_{out} = \left\lfloor \frac{W_{in} + 2P - K_w}{S} \right\rfloor + 1 \tag{5}$$

Trong ngữ cảnh OCR, các lớp tích chập đầu tiên sẽ học các đặc trưng cấp thấp (*low-level features*) như các cạnh (edges), góc (corners). Các lớp sâu hơn sẽ tổ hợp lại để nhận diện các nét bút (strokes) và cấu trúc tô pô của ký tự.



Hình 2: *Mô phỏng đơn giản về Convolution Layer*

2.2.2 Hàm kích hoạt phi tuyến (Non-linearity)

Sau mỗi phép tích chập, một hàm kích hoạt phi tuyến được áp dụng để mô hình có thể học được các mối quan hệ phức tạp. Hàm phổ biến nhất là **ReLU (Rectified Linear Unit)**, giúp giảm thiểu vấn đề biến mất đạo hàm (*vanishing gradient*) và tăng tốc độ hội tụ:

$$f(x) = \max(0, x) \tag{6}$$

Ngoài ReLU, có nhiều hàm activation khác được sử dụng trong các kiến trúc CNN hiện đại:

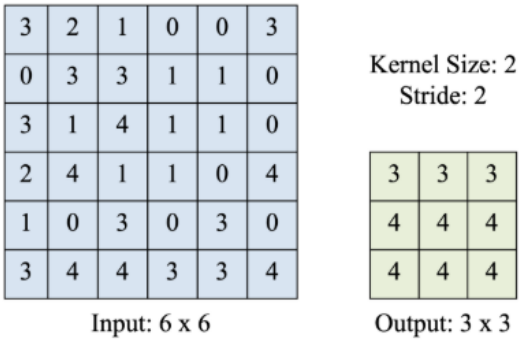
Bảng 1: *So sánh các hàm Activation phổ biến*

Hàm Activation	**Công thức**	**Ưu điểm/Nhược điểm**
ReLU	$f(x) = \max(0, x)$	Nhanh, tránh vanishing gradient, nhưng có dead neuron
Leaky ReLU	$f(x) = \max(\alpha x, x)$	Khắc phục dead neuron của ReLU
ELU	$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$	Smooth, tốt hơn ReLU nhưng chậm hơn
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	Dễ hiểu nhưng dễ bị vanishing gradient
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Đầu ra trong $[-1, 1]$, tốt hơn Sigmoid

2.2.3 Pooling và Bất biến dịch chuyển

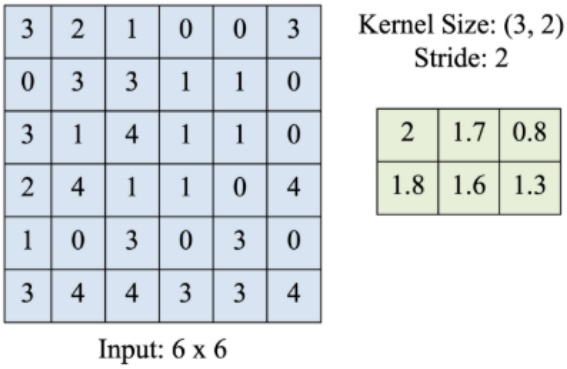
Lớp Pooling thực hiện giảm chiều dữ liệu (*down-sampling*) để giảm chi phí tính toán và tăng tính bất biến với các biến dạng nhẹ. Có hai loại phổ biến:

- **Max Pooling:** Chọn giá trị lớn nhất trong vùng lân cận (giữ lại đặc trưng nổi bật nhất).



Hình 3: *Mô phỏng đơn giản về Max Pooling*

- **Average Pooling:** Tính giá trị trung bình trong vùng lân cận (làm mượt đặc trưng).



Hình 4: *Mô phỏng đơn giản về Average Pooling*

Về mặt toán học, với một vùng lân cận $R_{i,j}$ kích thước $K \times K$ và stride S, công thức tính kích thước đầu ra sau Pooling (không padding) là:

$$H_{out} = \left\lfloor \frac{H_{in} - K}{S} \right\rfloor + 1 \tag{7}$$

Đối với OCR, lớp này giúp mô hình nhận diện được ký tự ngay cả khi nó bị dịch chuyển nhẹ hoặc co giãn trong ảnh (*Translation Invariance*).

Bảng 2: *So sánh các loại Pooling*

Loại Pooling	**Định nghĩa**	**Ứng dụng**
Max Pooling	$y_{i,j} = \max_{(p,q)\in R_{i,j}} x_{p,q}$	Giữ lại feature mạnh nhất
Average Pooling	$y_{i,j} = \frac{1}{\lvert R_{i,j}\rvert} \sum_{(p,q)\in R_{i,j}} x_{p,q}$	Làm mịn features
L2 Pooling	$y_{i,j} = \sqrt{\sum_{(p,q)\in R_{i,j}} x_{p,q}^2}$	Cân bằng hơn Max Pooling

2.3 Mô hình hóa chuỗi (Sequence Modeling) với RNN

Sau khi CNN trích xuất các đặc trưng hình ảnh, đầu ra là một chuỗi các vector đặc trưng $X = [x_1, x_2, ..., x_T]$, trong đó T là chiều dài của chuỗi (tương ứng với chiều rộng của ảnh sau khi downsampling). Tuy nhiên, CNN chỉ nắm bắt được thông tin cục bộ và thiếu khả năng xử lý ngữ cảnh chuỗi. Để giải quyết vấn đề này, Mạng Nơ-ron Hồi quy (RNN) được sử dụng.

2.4 Hạn chế của RNN truyền thống và giải pháp LSTM

Mạng RNN truyền thống cập nhật trạng thái ẩn h_t dựa trên đầu vào hiện tại x_t và trạng thái trước đó h_{t-1}:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b) \tag{8}$$

Tuy nhiên, RNN thường gặp vấn đề *biến mất đạo hàm* (vanishing gradient) khi xử lý các chuỗi dài, khiến nó không thể học được sự phụ thuộc xa. Để khắc phục, kiến trúc **Long Short-Term Memory (LSTM)** được đề xuất với cơ chế cổng (gating mechanism) bao gồm: cổng quên (f_t), cổng vào (i_t) và cổng ra (o_t), giúp điều tiết dòng thông tin qua các trạng thái tế bào (c_t).

3 Phân loại văn bản với RNN & Transformer

3.1 Tổng quan về Xử lý ngôn ngữ tự nhiên và Phân tích cảm xúc

Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing - NLP) là một nhánh của Trí tuệ Nhân tạo (AI) tập trung vào việc tương tác giữa máy tính và ngôn ngữ của con người. Trong giới hạn của nghiên cứu này, tác vụ cốt lõi được giải quyết là Phân tích cảm xúc (Sentiment Analysis) hay Phân loại văn bản (Text Classification).

Về mặt toán học, bài toán phân loại cảm xúc nhị phân có thể được định nghĩa như sau: Cho một tập hợp các văn bản (đánh giá phim) $D = \{d_1, d_2, \ldots, d_N\}$ và tập nhãn phân lớp $Y = \{0, 1\}$, trong đó 0 đại diện cho cảm xúc tiêu cực (Negative) và 1 đại diện cho cảm xúc tích cực (Positive). Mục tiêu của mô hình học máy là tìm ra một hàm ánh xạ $f : D \to Y$ sao cho hàm mục tiêu (thường là sự sai lệch giữa dự đoán và thực tế) đạt giá trị nhỏ nhất.

3.2 Cơ sở lý thuyết

3.2.1 Tiền xử lý dữ liệu và Không gian nhúng từ (Word Embeddings)

3.2.1.1 Tiền xử lý và Mã hóa (Tokenization)

Máy tính không thể trực tiếp xử lý văn bản dạng chuỗi ký tự. Do đó, dữ liệu thô cần được làm sạch (loại bỏ nhiễu như thẻ HTML, ký tự đặc biệt) và phân rã thành các đơn vị cơ sở gọi là *token* (từ, cụm từ, hoặc ký tự phụ). Mỗi token sau đó được ánh xạ thành một số nguyên duy nhất (ID) tạo thành một từ điển (Vocabulary). Tuy nhiên, việc biểu diễn bằng ID hay One-hot Encoding mang lại những ma trận thưa (sparse matrix) khổng lồ và hoàn toàn triệt tiêu mối quan hệ ngữ nghĩa giữa các từ.

3.2.1.2 Biểu diễn từ phân tán với GloVe (Global Vectors for Word Representation)

Để giải quyết hạn chế của One-hot Encoding, các kỹ thuật Word Embeddings ánh xạ mỗi từ vào một không gian vector thực liên tục (continuous vector space) nhiều chiều, nơi các từ có nghĩa tương đồng sẽ nằm gần nhau. Trong nghiên cứu này, thuật toán GloVe (cụ thể là ma trận GloVe 100 chiều) được sử dụng cho mô hình LSTM.

GloVe là một mô hình học không giám sát được phát triển bởi Đại học Stanford, kết hợp lợi thế của ma trận phân tích nhân tử toàn cục (global matrix factorization) và các phương pháp trượt cửa sổ ngữ cảnh cục bộ (local context window). Giả sử X là ma trận đồng xuất hiện (co-occurrence matrix), trong đó phần tử X_{ij} thể hiện số lần từ j xuất hiện trong ngữ cảnh của từ i. Gọi $X_i = \sum_k X_{ik}$ là tổng số lần bất kỳ từ nào xuất hiện trong ngữ cảnh của i. Xác suất đồng xuất hiện được định nghĩa là $P_{ij} = P(j|i) = \frac{X_{ij}}{X_i}$.

GloVe thiết lập một hàm mục tiêu dựa trên bình phương sai số có trọng số (weighted least squares) nhằm tối thiểu hóa sự chênh lệch giữa tích vô hướng của hai vector nhúng và logarit xác suất đồng xuất hiện của chúng:

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \tag{9}$$

Trong đó:

- V là kích thước tập từ vựng.
- $w_i, \tilde{w}_j \in \mathbb{R}^d$ là các vector biểu diễn của từ mục tiêu và từ ngữ cảnh.
- $b_i, \tilde{b}_j$ là các hệ số chệch (bias).

- $f(X_{ij})$ là hàm trọng số nhằm giới hạn sự ảnh hưởng của các từ xuất hiện quá thường xuyên và ngăn chặn hàm $\log(0)$ khi $X_{ij} = 0$.

3.2.2 Mạng Nơ-ron Hồi quy tiến hóa: Từ RNN đến Bi-LSTM

3.2.2.1 Mạng LSTM (Long Short-Term Memory)

Mạng Nơ-ron Hồi quy (RNN) truyền thống gặp phải bài toán triệt tiêu đạo hàm (Vanishing Gradient) khi chuỗi quá dài. LSTM giải quyết vấn đề này thông qua kiến trúc tế bào (cell) phức tạp chứa "trạng thái ô" (cell state - C_t), cùng với ba "cổng" (gates) để kiểm soát luồng thông tin:

- **Cổng quên (Forget gate - f_t):**

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{10}$$

- **Cổng đầu vào (Input gate - i_t) và Trạng thái ứng viên ($\tilde{C}_t$):**

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{11}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{12}$$

- **Cập nhật trạng thái ô (C_t):**

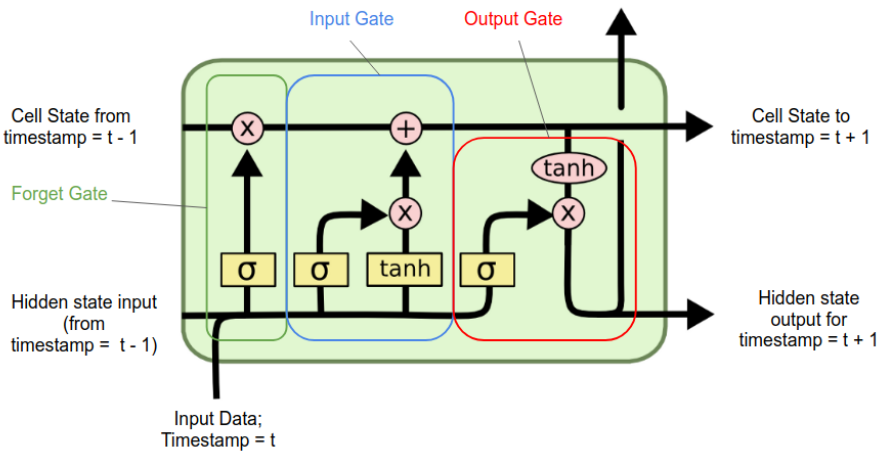
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{13}$$

- **Cổng đầu ra (Output gate - o_t) và Trạng thái ẩn (h_t):**

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{14}$$

$$h_t = o_t \odot \tanh(C_t) \tag{15}$$

(Trong đó: $\odot$ là ký hiệu cho phép nhân từng phần tử - element-wise multiplication).



Hình 5: *Cấu trúc chi tiết của một tế bào LSTM (LSTM cell)*

3.2.2.2 Mạng LSTM hai chiều (Bidirectional LSTM) và Cơ chế Gộp (Pooling)

Ngôn ngữ tự nhiên có tính phụ thuộc hai chiều. Bi-LSTM vận hành hai lớp LSTM song song: chiều xuôi ($\overrightarrow{h_t}$) và chiều ngược ($\overleftarrow{h_t}$). Trạng thái ẩn tổng hợp là sự ghép nối:

$$h_t = [\overrightarrow{h_t} \oplus \overleftarrow{h_t}] \tag{16}$$

Để trích xuất đặc trưng toàn cục, mô hình sử dụng kết hợp hai phép toán Pooling:

- **Average Pooling (h_{avg}):** Nắm bắt ngữ nghĩa tổng thể.
- **Max Pooling (h_{max}):** Phát hiện các từ khóa mang ý nghĩa quyết định.

Vector đặc trưng cuối cùng: $v_{out} = [h_{avg} \oplus h_{max}]$.

3.2.3 Kiến trúc Transformer và Mô hình BERT

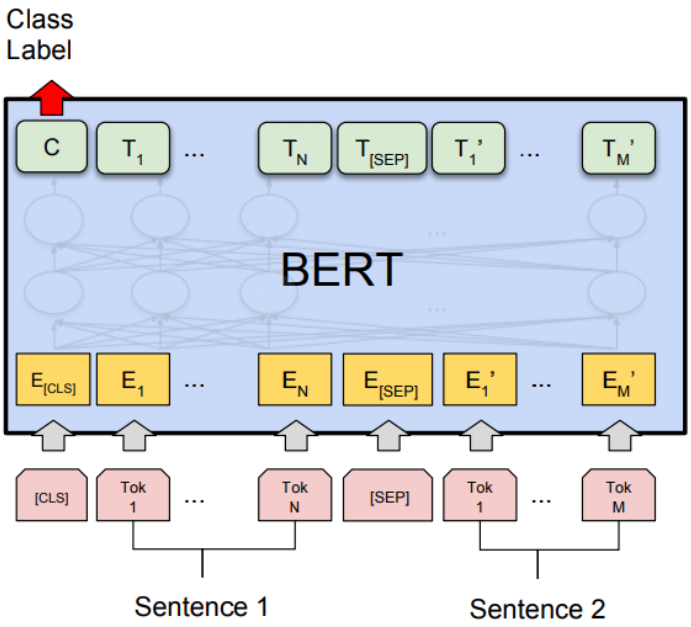
3.2.3.1 Cơ chế Tự chú ý (Self-Attention)

Cơ chế này cho phép mô hình tính toán mức độ quan tâm của một từ đến tất cả các từ khác trong câu thông qua ba vector: Query (Q), Key (K) và Value (V):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{17}$$

3.2.3.2 Mô hình BERT

BERT (Bidirectional Encoder Representations from Transformers) dựa trên các khối Encoder của Transformer. Nghiên cứu áp dụng phiên bản `bert-base-uncased` (12 layers, hidden size 768). Đồ án sử dụng phương pháp Học chuyển giao (Transfer Learning) bằng cách Tinh chỉnh (Fine-tuning) lớp phân loại trên token đặc biệt `[CLS]`.



Hình 6: *Khái quát kiến trúc mô hình BERT*

3.2.4 Hàm mất mát và Kỹ thuật Tối ưu hóa

3.2.4.1 Hàm mất mát (Loss Functions)

Đối với bài toán phân loại nhị phân, hàm Binary Cross-Entropy Loss được áp dụng:

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\sigma(\hat{y}_i)) + (1 - y_i)\log(1 - \sigma(\hat{y}_i))\right] \tag{18}$$

3.2.4.2 Kỹ thuật chống Quá khớp (Overfitting)

- **Dropout:** Vô hiệu hóa ngẫu nhiên nơ-ron (tỷ lệ $p = 0.3$).
- **Gradient Clipping:** Giới hạn max_norm $= 1.0$ để ngăn bùng nổ đạo hàm.
- **Tối ưu hóa:** Sử dụng thuật toán **AdamW** (cho BERT) và **Adam** (cho Bi-LSTM).

3.2.5 Các độ đo đánh giá (Evaluation Metrics)

Dựa trên ma trận nhầm lẫn (Confusion Matrix) với các giá trị TP, TN, FP, FN, các chỉ số được xác định như sau:

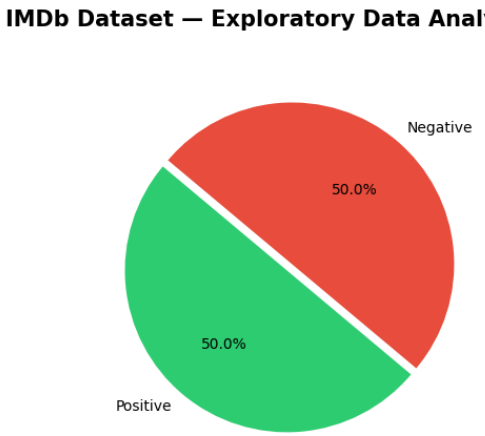
- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** $\frac{TP}{TP+FP}$
- **Recall:** $\frac{TP}{TP+FN}$
- **F1-Score:** $2 \times \frac{Precision \times Recall}{Precision + Recall}$

3.3 Mô hình đề xuất

Để giải quyết bài toán phân loại cảm xúc trên văn bản một cách hiệu quả, nghiên cứu này đề xuất và đối sánh hai quy trình hoàn chỉnh (end-to-end pipelines): một phương pháp dựa trên kiến trúc hồi quy truyền thống (Bi-LSTM kết hợp GloVe) và một phương pháp dựa trên học chuyển giao (Fine-tuning BERT). Chương này trình bày chi tiết về luồng xử lý dữ liệu, kiến trúc mạng nơ-ron và các chiến lược tối ưu hóa.

3.3.1 Phân tích Khám phá và Phân chia Dữ liệu (EDA & Data Splitting)

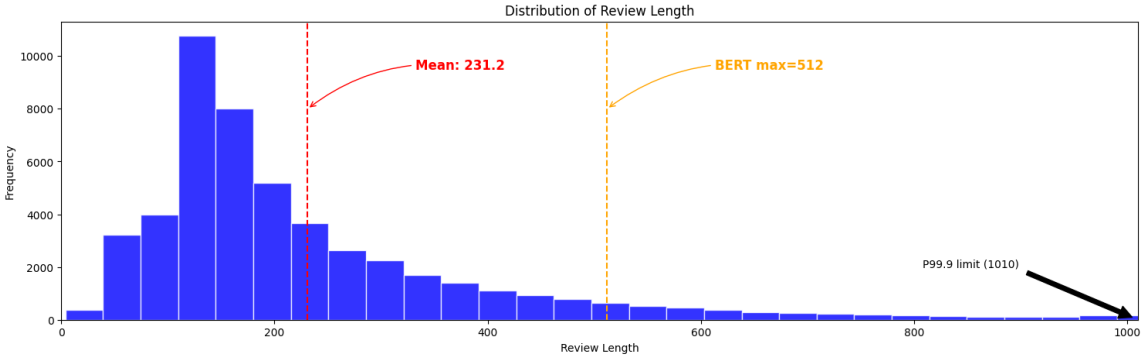
Tập dữ liệu được sử dụng là *IMDb Reviews Dataset*, bao gồm 50,000 đánh giá điện ảnh được dán nhãn nhị phân. Tập dữ liệu thể hiện tính cân bằng hoàn hảo với tỷ lệ 50% cho mỗi lớp (Tích cực và Tiêu cực), giúp loại bỏ rủi ro thiên lệch (class imbalance bias).



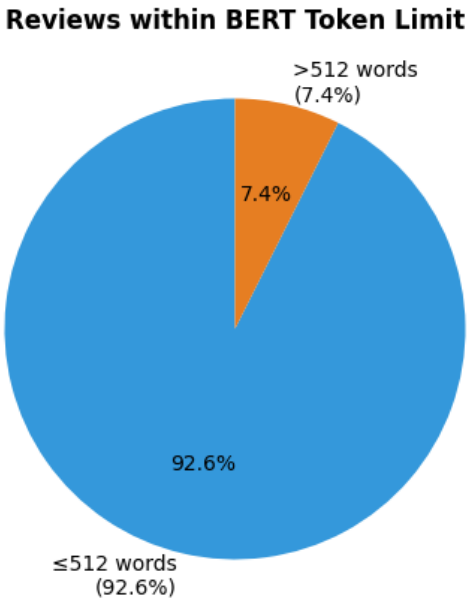
Hình 7: *Phân phối nhãn tập dữ liệu*

Hình 8: *Từ khóa phổ biến trong các đánh giá*

Phân tích phân phối chỉ ra rằng 92.6% số lượng đánh giá có độ dài ≤ 512 từ. Do đó, ngưỡng giới hạn chiều dài chuỗi (*max sequence length*) được thiết lập như sau:



Hình 9: *Phân phối chiều dài các đánh giá trong tập dữ liệu*



Hình 10: *Tỷ lệ đánh giá nằm trong giới hạn 512 từ của mô hình BERT*

- **Mô hình BERT:** 512 token (tối đa hóa ngữ nghĩa).
- **Mô hình Bi-LSTM:** 256 token (tối ưu hóa tốc độ tính toán).

Dữ liệu được phân chia theo chiến lược phân tầng (*stratified split*) với tỷ lệ 80-10-10:

Tập dữ liệu	Số lượng mẫu	Mục đích
Training Set	$40,000$	Cập nhật trọng số mạng.
Validation Set	$5,000$	Tinh chỉnh siêu tham số và lưu checkpoint.
Test Set	$5,000$	Đánh giá độ tổng quát hóa cuối cùng.

Bảng 3: *Phân chia tập dữ liệu nghiên cứu*

3.3.2 Tiền xử lý Văn bản và Biểu diễn Dữ liệu

Nghiên cứu xây dựng quy trình chuẩn hóa văn bản thông qua hàm `clean_text` sử dụng biểu thức chính quy (Regex) để: chuyển chữ thường, loại bỏ thẻ HTML (ví dụ: `<br />`), tách dấu câu và chuẩn hóa khoảng trắng.

3.3.2.1 Biểu diễn cho mô hình Bi-LSTM

Văn bản được tách từ dựa trên khoảng trắng và ánh xạ vào ma trận nhúng **GloVe 100 chiều** (`glove.6B.100d.txt`).

- Kích thước từ vựng thực tế: $111,798$ token.
- Tỷ lệ bao phủ (*Coverage*): 66.40%.
- Các chuỗi ngắn hơn 256 được đệm (*padding*) bằng ID 0.

3.3.2.2 Biểu diễn cho mô hình BERT

Sử dụng bộ `BertTokenizer` từ kiến trúc `bert-base-uncased` với thuật toán **WordPiece**. Kích thước từ vựng đạt $30,522$ token. Các chuỗi được xử lý tự động để đạt độ dài cố định `MAX_LEN = 512`.

3.3.3 Kiến trúc Đề xuất 1: Mạng Bi-LSTM tích hợp Gộp đặc trưng (Pooling)

Mô hình `LSTMClassifier` bao gồm các tầng chức năng sau:

1. **Embedding Layer:** Nhận đầu vào kích thước ($B \times 256$), ánh xạ qua trọng số GloVe đã được đóng băng (`freeze=True`).
2. **Bi-LSTM Layer:** Kích thước trạng thái ẩn (*hidden size*) là 128. Đầu ra tại mỗi bước thời gian t là $h_t \in \mathbb{R}^{256}$ (do kết hợp hai chiều).
3. **Multi-pooling Strategy:** Trích xuất đặc trưng toàn cục bằng cách nối kết (*concatenate*) kết quả của Average Pooling và Max Pooling:

$$v_{concat} = \left[\frac{1}{L}\sum_{t=1}^{L} h_t\right] \oplus \left[\max_{t=1}^{L} h_t\right] \tag{19}$$

   Kết quả thu được vector đặc trưng 512 chiều.
4. **Classification Head:** Đi qua lớp Dropout ($p = 0.3$) và một tầng tuyến tính (`nn.Linear(512, 1)`).

3.3.4 Kiến trúc Đề xuất 2: Tinh chỉnh Mô hình BERT (Fine-tuning)

Nghiên cứu sử dụng biến thể `bert-base-uncased` (12 layers, 12 heads, 768 hidden size). Chiến lược **Học chuyển giao** được thực hiện bằng cách thêm một lớp phân loại tuyến tính lên trên token đầu ra `[CLS]`. Toàn bộ 109 triệu tham số sẽ được cập nhật nhẹ trong quá trình Fine-tuning.

3.3.5 Cấu hình Huấn luyện và Chiến lược Tối ưu hóa

3.3.5.1 Tối ưu hóa Mô hình Bi-LSTM

- **Hàm mất mát:** Binary Cross Entropy with Logits.
- **Optimizer:** Adam ($\eta = 10^{-3}$).
- **Batch size:** 16; **Epochs:** 100.
- **Gradient Clipping:** `max_norm=1.0`.

3.3.5.2 Tối ưu hóa Mô hình BERT

Quá trình Fine-tuning áp dụng các kỹ thuật đặc thù:

- **Optimizer:** AdamW ($\eta = 2 \times 10^{-5}$, $\lambda = 0.01$).
- **Scheduler:** *Linear Schedule with Warmup*. Tốc độ học tăng dần trong 10% số bước đầu tiên (*warmup ratio* $= 0.1$) sau đó giảm dần về 0.
- **Quy mô:** 3 epoch (tương đương 7,500 bước huấn luyện).

Thông số	Bi-LSTM + GloVe	BERT (Fine-tuning)
Learning Rate	10^{-3}	2×10^{-5}
Optimizer	Adam	AdamW
Max Length	256	512
Batch Size	16	16
Epochs	100 (Early Stopping)	3

Bảng 4: *So sánh cấu hình huấn luyện của hai mô hình*

3.3.5.3 Tóm tắt siêu tham số thực nghiệm

3.4 Kết quả Thực nghiệm và Đánh giá

Phần này trình bày chi tiết các kết quả đạt được sau quá trình huấn luyện hai kiến trúc mạng học sâu: Bi-LSTM kết hợp không gian nhúng GloVe và mô hình ngôn ngữ lớn BERT. Các phân tích được thực hiện dựa trên sự biến thiên của hàm mất mát, các độ đo phân loại chuẩn mực trên tập kiểm thử (Test Set) và đánh giá định tính qua các ca kiểm thử thực tế (Inference).

3.4.1 Môi trường Thực nghiệm và Cấu hình Siêu tham số

Nhằm đảm bảo tính công bằng và khả năng tái lập kết quả (reproducibility), toàn bộ các thực nghiệm được tiến hành trên cùng một môi trường phần cứng và phần mềm:

- **Tài nguyên điện toán:** Bộ xử lý đồ họa (GPU) Tesla T4 với dung lượng bộ nhớ khả dụng xấp xỉ 15.6 GB.
- **Cấu hình BERT:** Sử dụng biến thể `bert-base-uncased`. Thuật toán tối ưu AdamW được áp dụng với tốc độ học $\eta = 2 \times 10^{-5}$, hệ số suy giảm trọng số (weight decay) 0.01 và tỷ lệ hâm nóng (warmup ratio) 0.1. Quy mô lô (batch size) là 16. Quá trình tinh chỉnh (fine-tuning) diễn ra trong 3 epoch (tương đương 7,500 bước huấn luyện).
- **Cấu hình Bi-LSTM:** Đầu vào được giới hạn 256 token, đi qua ma trận nhúng GloVe 100 chiều. Thuật toán tối ưu Adam với $\eta = 10^{-3}$, batch size 16. Quá trình huấn luyện được thiết lập chạy tối đa 100 epoch để quan sát rõ ranh giới hội tụ và quá khớp.

3.4.2 Phân tích Động lực học Quá trình Huấn luyện

Sự biến thiên của hàm suy hao (Loss) và độ chính xác (Accuracy) qua các kỷ nguyên là cơ sở để đánh giá khả năng tổng quát hóa của mô hình.

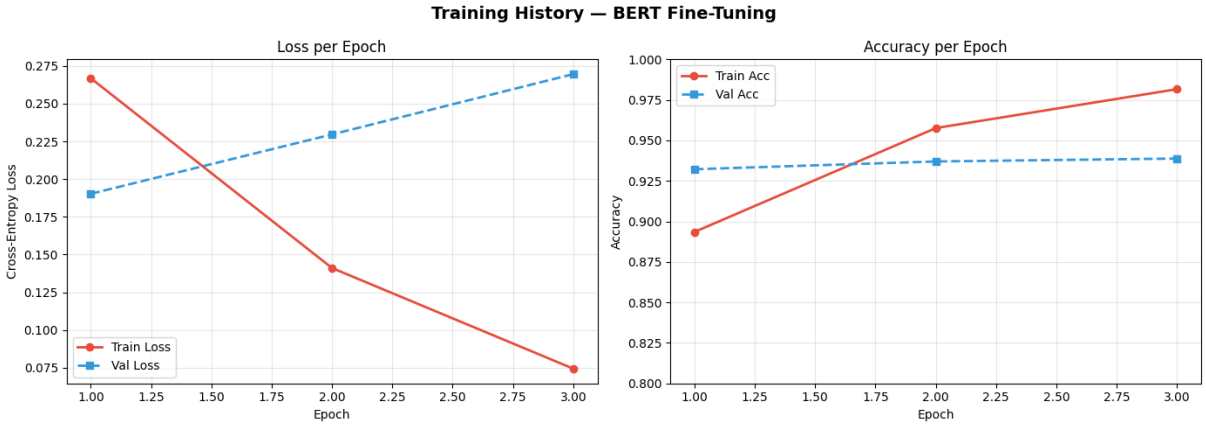
3.4.2.1 Động lực học của Mô hình BERT

Quá trình tinh chỉnh BERT cho thấy sự hội tụ cực kỳ nhanh chóng. Ngay tại Kỷ nguyên 1 (Epoch 1), hàm mất mát trên tập huấn luyện (Train Loss) đã giảm xuống mức 0.2669, đồng thời độ chính xác (Train Acc) đạt 89.35%. Tương ứng, độ chính xác trên tập kiểm định (Validation) đạt 93.22%.

Tuy nhiên, biểu đồ huấn luyện cho thấy dấu hiệu của hiện tượng quá khớp vi mô (micro-overfitting) bắt đầu xuất hiện từ Kỷ nguyên 2:

- Train Loss tiếp tục giảm mạnh xuống 0.0744 và Train Acc đạt đỉnh 98.16%.
- Ngược lại, Validation Loss bắt đầu tăng ngược trở lại lên mức 0.2696, dù Validation Acc vẫn cải thiện nhẹ lên 93.88%.

Hệ thống đã tự động lưu lại điểm kiểm tra (checkpoint) tại Kỷ nguyên 2 với chỉ số $Best\ Val\ F1 = 0.9388$ làm mô hình dự đoán chính thức.



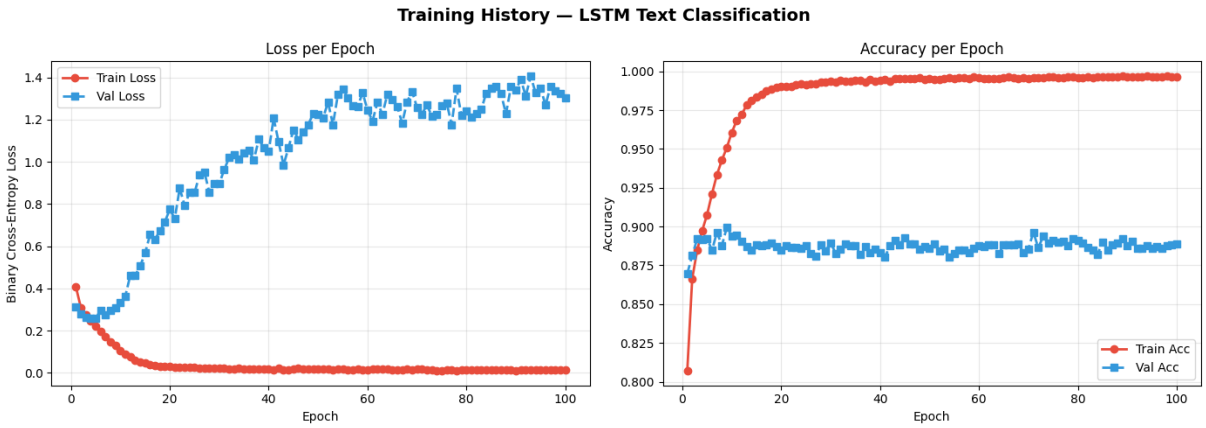
Hình 11: *Đồ thị hàm mất mát và độ chính xác của mô hình BERT qua 3 epoch*

3.4.2.2 Động lực học của Mô hình Bi-LSTM

Trái ngược với khả năng hội tụ ổn định của BERT, Bi-LSTM bộc lộ rõ rệt đặc tính khó kiểm soát của mạng hồi quy khi đối mặt với dữ liệu nhiễu trong thời gian dài.

- Xuyên suốt 100 kỷ nguyên, Train Loss của LSTM tiệm cận về 0 (đạt 0.0146 ở Epoch 100) và Train Acc gần như hoàn hảo ở mức 99.62%.
- Dẫu vậy, bắt đầu từ khoảng Epoch 10-15, Validation Loss bắt đầu phân kỳ và tăng theo cấp số nhân, đạt mức 1.3039 ở Epoch cuối cùng, kéo theo Validation Acc suy giảm xuống 88.90%.

Sự phân kỳ sâu sắc này là minh chứng toán học kinh điển cho hiện tượng quá khớp (Overfitting) nghiêm trọng. Mạng LSTM, do hạn chế về dung lượng bộ nhớ ngữ cảnh xa, đã chọn cách "học vẹt" các đặc trưng cục bộ của tập huấn luyện thay vì rút ra quy luật ngôn ngữ tổng quát. Điểm $Best\ Val\ F1$ tối ưu nhất được ghi nhận là 0.8994 trước khi mô hình trượt dài vào vùng quá khớp.



Hình 12: *Động lực học của mô hình Bi-LSTM trong suốt 100 epoch với hiện tượng quá khớp*

3.4.3 Đánh giá Định lượng trên Tập Kiểm thử (Test Set)

Để đưa ra kết luận khách quan, hai mô hình tốt nhất (Best Checkpoints) được tải lên và suy diễn trên tập Test hoàn toàn độc lập gồm 5,000 mẫu (2,500 Tiêu cực và 2,500 Tích cực).

Bảng 5: *Báo cáo phân loại (Classification Report) chi tiết của hai mô hình*

Chỉ số	Lớp (Class)	Bi-LSTM	BERT (Fine-tuned)
Độ xác thực (Precision)	Negative (0)	0.90	**0.95**
	Positive (1)	0.89	**0.93**
Độ bao phủ (Recall)	Negative (0)	0.89	**0.93**
	Positive (1)	0.90	**0.95**
F1-Score	Negative (0)	0.90	**0.94**
	Positive (1)	0.89	**0.94**

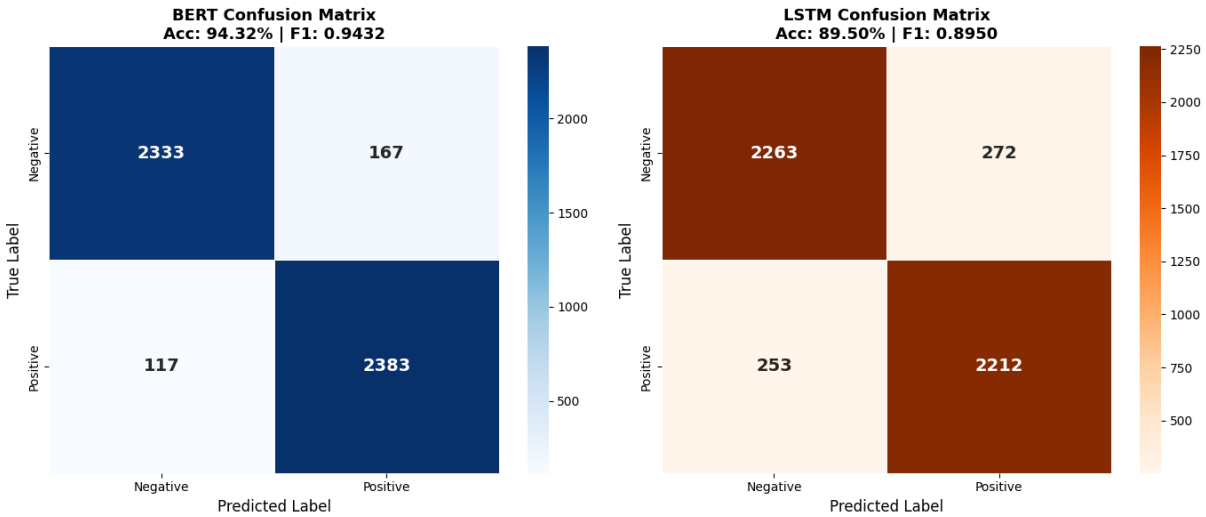
Bảng 6: *Tổng hợp hiệu suất tổng thể (Overall Performance)*

Độ đo (Metric)	Bi-LSTM	BERT (Fine-tuned)	Khoảng cách
Accuracy (Độ chính xác)	89.50%	**94.32%**	+4.82%
Test Loss	0.3089	**0.2511**	−0.0578
Macro Avg F1-Score	0.8950	**0.9432**	+0.0482

Dữ liệu từ hai bảng trên khẳng định sự ưu việt toàn diện của kiến trúc Transformer. BERT đạt mức F1-Score đồng đều 0.94 cho cả hai lớp cảm xúc, trong khi Độ xác thực (Precision) của lớp Negative lên tới 0.95, đồng nghĩa với việc khi BERT dự đoán một đánh giá là "Tiêu cực", khả năng sai sót chỉ ở mức 5%. Bi-LSTM duy trì mức hiệu suất dao động quanh biên độ $0.89 - 0.90$, một kết quả đáng khích lệ đối với kiến trúc tuần tự nhưng vẫn bộc lộ khoảng cách công nghệ rõ nét.

3.4.4 Phân tích Ma trận Nhầm lẫn (Confusion Matrix)

Phân tích 4 thành phần TP, TN, FP, FN cung cấp góc nhìn vi mô về các điểm mù của thuật toán.



Hình 13: *Ma trận nhầm lẫn (Confusion Matrix) của hai mô hình trên tập kiểm thử*

1. **Mô hình BERT:**
   - True Negatives (TN): $2,333$ mẫu.
   - True Positives (TP): $2,383$ mẫu.
   - False Positives (FP): 167 mẫu.
   - False Negatives (FN): 117 mẫu.

   Phân phối lỗi của BERT khá cân bằng và được nén ở mức tối thiểu. Việc FN (117) thấp hơn FP (167) cho thấy BERT có xu hướng "nhạy" hơn một chút trong việc phát hiện cảm xúc tích cực, đôi khi dán nhãn nhầm các câu chê bai tinh vi thành khen ngợi.

2. **Mô hình Bi-LSTM:**
   - True Negatives (TN): $2,263$ mẫu.
   - True Positives (TP): $2,212$ mẫu.
   - False Positives (FP): 272 mẫu.
   - False Negatives (FN): 253 mẫu.

   Tổng số mẫu bị phân loại sai của LSTM là 525 mẫu, cao hơn 84.8% so với tổng số lỗi của BERT (284 mẫu). LSTM gặp khó khăn lớn đối với các cấu trúc ngữ pháp đảo ngược hoặc châm biếm. Khi khoảng cách vật lý giữa hư từ (như "not", "never") và tính từ gốc (như "good", "masterpiece") quá xa, lớp RNN dần quên đi bối cảnh phủ định, dẫn đến các sai số FP và FN gia tăng mạnh. Cơ chế Self-Attention của BERT đã khắc phục triệt để nhược điểm này bằng cách tính toán trọng số tương quan trực tiếp giữa mọi cặp từ bất chấp khoảng cách.

3.4.5 Đánh giá Định tính: Thử nghiệm Suy diễn (Inference Case Studies)

Để kiểm chứng năng lực ngữ nghĩa thực tế, 5 mẫu văn bản giả định mang các sắc thái cực đoan và hỗn hợp được đưa vào hệ thống suy diễn (inference pipeline) để quan sát sự tự tin (confidence scores) của hai mô hình.

- **Ca kiểm thử số 1 (Tích cực mạnh):** *"An absolute masterpiece. The cinematography is stunning..."* Cả hai mô hình đều trích xuất thành công các tính từ mạnh ("masterpiece", "stunning"). BERT tự tin dự đoán POSITIVE với 99.9%, LSTM tự tin tuyệt đối 100.0%.
- **Ca kiểm thử số 2 (Tiêu cực mạnh):** *"What a complete waste of time and money..."* Các ngữ nghĩa "waste", "wooden" giúp hai mô hình dễ dàng dán nhãn NEGATIVE với sự tự tin cực đại (BERT 99.9%, LSTM 100.0%).
- **Ca kiểm thử số 3 (Cảm xúc hỗn hợp/Phân cực chìm):** *"It was okay, nothing special. Some scenes were interesting but the pacing felt off. Probably wouldn't watch it again."*
  Đây là một mẫu văn bản bẫy thuật toán (adversarial example) khi đan xen các từ có vẻ tích cực ("okay", "interesting") bên trong một đánh giá tiêu cực tổng thể.
  - Mô hình BERT xử lý xuất sắc, triệt tiêu nhiễu từ các từ tích cực và dán nhãn NEGATIVE với xác suất 99.8%.
  - Mô hình Bi-LSTM, do bị tác động bởi các hàm lượng thông tin tích cực xuất hiện rải rác, đã bị giảm độ tự tin xuống còn 97.8%, phân bổ 2.2% xác suất cho lớp POSITIVE. Sự dao động này phản ánh hạn chế trong việc tổng hợp ngữ nghĩa toàn cục của mạng quy hồi.
- **Ca kiểm thử số 4 và 5:** Cả hai mô hình tiếp tục thể hiện sự ổn định khi nhận diện chính xác câu khen ngợi đạo diễn Nolan (POSITIVE) và câu chê bai CGI tệ hại (NEGATIVE) với độ tin cậy gần như tuyệt đối.

3.4.6 Tiểu kết

Kết quả thực nghiệm đã minh chứng định lượng sức mạnh biểu diễn vượt trội của mô hình BERT. Bằng cách tận dụng cơ chế Tự chú ý và không gian kiến thức học chuyển giao, BERT không chỉ thiết lập mức hiệu suất 94.32% mà còn duy trì độ bền vững tuyệt đối trước các văn bản có cấu trúc đánh lừa. Về phần mình, kiến trúc Bi-LSTM với ma trận GloVe, dù có hiện tượng quá khớp và độ chính xác thấp hơn (89.50%), vẫn chứng minh được giá trị thực tiễn trong các bài toán yêu cầu chi phí tham số thấp và tốc độ suy diễn nhanh.

4 Phân loại Đa phương thức với CLIP

4.1 Giới thiệu

Phân loại đa phương thức (Multimodal Classification) là bài toán nhận dạng và phân loại dữ liệu từ nhiều nguồn thông tin khác nhau, trong đó phổ biến nhất là sự kết hợp giữa **hình ảnh** và **văn bản**.

Khác với phân loại đơn phương thức truyền thống (chỉ dùng ảnh hoặc chỉ dùng text), phương pháp đa phương thức có khả năng tận dụng thông tin bổ sung từ nhiều modality để cải thiện độ chính xác.

Trong phần này, chúng tôi thực hiện so sánh **ba phương pháp** phân loại đa phương thức trên tập dữ liệu COCO (Common Objects in Context), sử dụng mô hình CLIP (Contrastive Language-Image Pre-training) từ OpenAI:

1. **Zero-shot Classification**: Phân loại trực tiếp mà không cần huấn luyện thêm.
2. **Few-shot Classification (Frozen Backbone)**: Tinh chỉnh đầu phân loại với số lượng mẫu nhỏ (k=1,5,10,20), giữ nguyên trọng số của mạng nền CLIP.
3. **Full Fine-Tuning (Unfrozen Backbone)**: Tinh chỉnh toàn bộ mô hình CLIP với lượng dữ liệu lớn hơn (100 samples/class).

4.2 Tập dữ liệu COCO 2017

4.2.1 Giới thiệu về COCO

COCO (Common Objects in Context) là một trong những tập dữ liệu chuẩn quốc tế quan trọng nhất cho các bài toán thị giác máy tính, bao gồm:

- **Object Detection** (Phát hiện đối tượng)
- **Instance Segmentation** (Phân đoạn thực thể)
- **Image Captioning** (Mô tả ảnh)
- **Multimodal Classification** (Phân loại đa phương thức)

Tập dữ liệu gốc chứa **80 categories** phổ biến trong đời sống hàng ngày, từ con người, phương tiện, động vật đến đồ vật gia dụng. Mỗi ảnh trong COCO đi kèm với:

- **Annotations**: Bounding boxes, segmentation masks
- **Captions**: 5 câu mô tả tiếng Anh cho mỗi ảnh
- **Category labels**: Nhãn phân loại đối tượng

4.2.2 Xử lý và Lọc Dữ liệu

Trong thực nghiệm này, chúng tôi sử dụng phiên bản COCO 2017 từ Kaggle[1]. Quy trình xử lý dữ liệu bao gồm:

1. **Kết hợp train và validation splits**: Gộp cả train2017 và val2017 để tăng số lượng mẫu.
2. **Lọc categories thiếu dữ liệu**: Loại bỏ các categories có ít hơn 100 samples để đảm bảo chất lượng huấn luyện.
3. **Primary category assignment**: Với mỗi ảnh chứa nhiều đối tượng, chọn đối tượng có diện tích lớn nhất làm category chính.
4. **Caption extraction**: Lấy caption đầu tiên cho mỗi ảnh làm thông tin văn bản.

Sau khi xử lý, tập dữ liệu cuối cùng có các thông số sau:

Bảng 7: *Thống kê tập dữ liệu COCO sau xử lý*

Thuộc tính	**Giá trị**
Tổng số categories	43
Tổng số samples	4,424
Samples per class	100-103
Training set (80%)	3,539
Test set (20%)	885
Modality	Image + Text (caption)

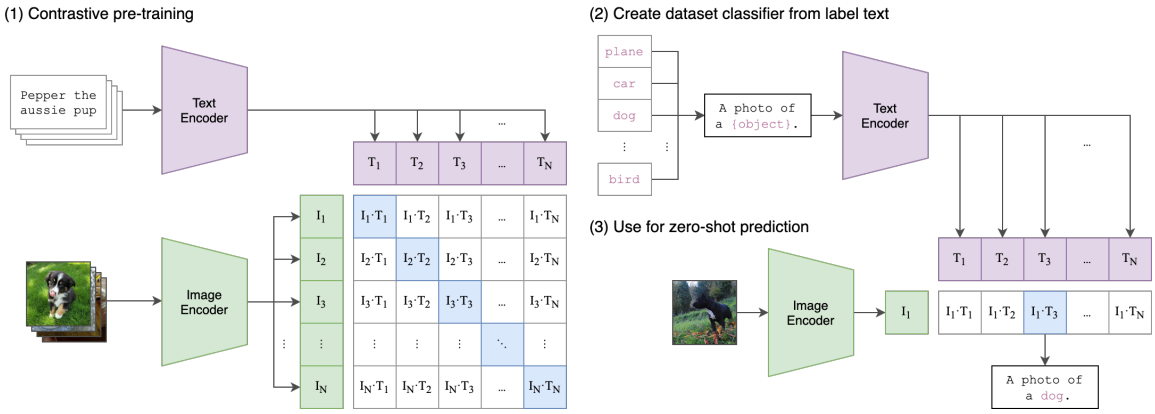
[1]Kaggle COCO Dataset: https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset

43 categories được giữ lại bao gồm: person, dog, bowl, bench, toilet, motorcycle, boat, bed, giraffe, car, zebra, chair, airplane, umbrella, bus, cow, dining table, teddy bear, couch, horse, bird, elephant, sheep, clock, train, bicycle, truck, oven, refrigerator, pizza, stop sign, kite, sink, sandwich, cat, fire hydrant, tv, traffic light, bear, suitcase, laptop, surfboard, vase.

4.3 Kiến trúc CLIP

4.3.1 Tổng quan về CLIP

CLIP (Contrastive Language-Image Pre-training) là mô hình đột phá từ OpenAI, được giới thiệu năm 2021[1]. Điểm đặc biệt của CLIP là khả năng học biểu diễn chung (joint representation) cho cả hình ảnh và văn bản thông qua huấn luyện contrastive trên **400 triệu cặp image-text** từ internet.



Hình 14: *Kiến trúc CLIP: Vision Encoder và Text Encoder được huấn luyện để tạo ra biểu diễn tương đồng cho các cặp ảnh-văn bản phù hợp*[2]

4.3.2 Các thành phần chính

CLIP bao gồm hai encoder hoạt động song song:

4.3.2.1 Vision Encoder Sử dụng kiến trúc **Vision Transformer (ViT)** với patch size 32x32. Mô hình chia ảnh đầu vào thành các patches nhỏ, sau đó xử lý chúng như một chuỗi tokens tương tự như trong NLP.

- **Input**: Ảnh RGB kích thước 224×224 pixels
- **Patch embedding**: Chia thành $7 \times 7 = 49$ patches
- **Transformer layers**: 12 layers với multi-head attention
- **Output**: Image embedding vector $\in \mathbb{R}^{512}$

4.3.2.2 Text Encoder Sử dụng kiến trúc **Transformer** để mã hóa câu văn bản thành vector đặc trưng.

- **Input**: Text sequence (max 77 tokens)
- **Tokenization**: Byte-Pair Encoding (BPE)
- **Transformer layers**: 12 layers với causal attention
- **Output**: Text embedding vector $\in \mathbb{R}^{512}$

[2]Nguồn: OpenAI CLIP repository - https://github.com/openai/CLIP

4.3.3 Contrastive Learning

CLIP được huấn luyện với mục tiêu contrastive: tối đa hóa cosine similarity giữa các cặp (image, text) đúng và tối thiểu hóa similarity giữa các cặp không khớp. Hàm loss được định nghĩa:

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left(\mathcal{L}_{\text{I2T}} + \mathcal{L}_{\text{T2I}}\right) \tag{20}$$

Trong đó:

- $\mathcal{L}_{\text{I2T}}$: Cross-entropy loss từ image sang text
- $\mathcal{L}_{\text{T2I}}$: Cross-entropy loss từ text sang image

4.3.4 Tại sao CLIP phù hợp cho Multimodal Classification?

1. **Pre-trained trên dữ liệu đa dạng**: 400M cặp image-text từ internet.
2. **Zero-shot capability**: Có thể phân loại các class chưa từng thấy trong quá trình huấn luyện.
3. **Joint embedding space**: Biểu diễn ảnh và text trong cùng một không gian vector.
4. **Transfer learning friendly**: Dễ dàng fine-tune cho các tác vụ downstream.

4.4 Ba Phương pháp So sánh

4.4.1 Zero-Shot Classification

4.4.1.1 Khái niệm Zero-shot classification là phương pháp phân loại **không cần huấn luyện thêm** (no additional training). Mô hình CLIP pretrained được sử dụng trực tiếp để phân loại các ảnh bằng cách so sánh chúng với các text prompts tương ứng với mỗi category.

4.4.1.2 Cơ chế hoạt động

1. **Tạo text prompts**: Với mỗi category c, tạo prompt theo template: `"a photo of a {category}"` Ví dụ:
   - Category "person" → `"a photo of a person"`
   - Category "car" → `"a photo of a car"`
2. **Encode image và text**:

$$\mathbf{v}_{\text{img}} = \text{VisionEncoder}(\text{image}) \tag{21}$$

$$\mathbf{v}_{\text{text}}^{(i)} = \text{TextEncoder}(\text{prompt}_i), \quad i = 1, \ldots, C \tag{22}$$

3. **Tính similarity scores**:

$$s_i = \cos(\mathbf{v}_{\text{img}}, \mathbf{v}_{\text{text}}^{(i)}) = \frac{\mathbf{v}_{\text{img}} \cdot \mathbf{v}_{\text{text}}^{(i)}}{\|\mathbf{v}_{\text{img}}\| \|\mathbf{v}_{\text{text}}^{(i)}\|} \tag{23}$$

4. **Dự đoán category**:

$$\hat{y} = \arg\max_i \text{softmax}(s_i/\tau) \tag{24}$$

   Trong đó τ là temperature parameter (thường $= 0.01$).

4.4.1.3 Ưu điểm

- Không cần dữ liệu huấn luyện
- Thời gian inference nhanh (không có training phase)
- Có thể mở rộng sang các class mới ngay lập tức
- Tận dụng tối đa knowledge từ pretrained model

4.4.1.4 Nhược điểm

- Không customize được cho dataset cụ thể
- Performance bị giới hạn bởi pretrained knowledge
- Có thể kém hiệu quả với domain-specific tasks

Bảng 8: *Thông số Zero-shot Classification*

Tham số	Giá trị
Model	openai/clip-vit-base-patch32
Training data	0 samples
Trainable parameters	0 (all frozen)
Training time	0s
Prompt template	"a photo of a {category}"
Temperature τ	0.01

4.4.1.5 Cấu hình thực nghiệm

4.4.2 Few-Shot Classification (Frozen Backbone)

4.4.2.1 Khái niệm Few-shot classification là phương pháp huấn luyện mô hình với **số lượng mẫu rất nhỏ** cho mỗi class (thường từ 1-20 samples). Trong phương pháp này, backbone CLIP được **đóng băng** (frozen), chỉ có classification head được huấn luyện.

4.4.2.2 Kiến trúc Mô hình bao gồm hai phần chính:

1. **CLIP Encoder (FROZEN)**:
   - Vision Encoder: xử lý ảnh → image embedding
   - Text Encoder: xử lý caption → text embedding
   - Tất cả trọng số của CLIP đều `requires_grad=False`
2. **Classification Head (TRAINABLE)**:
   - Input: Concatenation của image embedding và text embedding
   - Architecture: 2-layer MLP
   - Layer 1: Linear($512 \times 2 = 1024$, 512) + ReLU + Dropout(0.1)
   - Layer 2: Linear(512, num_classes)
   - Output: Logits cho 43 categories

```python
class CLIPClassifier(nn.Module):
    def __init__(self, clip_model, num_classes, freeze_backbone=True):
        super().__init__()
        self.clip = clip_model

        # Đóng băng CLIP backbone
        if freeze_backbone:
            for param in self.clip.parameters():
                param.requires_grad = False

        # Classification head (trainable)
        hidden_size = self.clip.config.projection_dim  # 512
        self.classifier = nn.Sequential(
            nn.Linear(hidden_size * 2, hidden_size),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(hidden_size, num_classes)
```

```
        )

    def forward(self, input_ids, attention_mask, pixel_values):
        outputs = self.clip(input_ids, attention_mask, pixel_values)
        combined = torch.cat([outputs.image_embeds,
                              outputs.text_embeds], dim=1)
        return self.classifier(combined)
```

4.4.2.3 Cấu hình huấn luyện Chúng tôi thực nghiệm với **4 giá trị k khác nhau**: k=1, 5, 10, 20 samples per class.

Bảng 9: *Thông số Few-shot Classification*

Tham số	**1-shot**	**5-shot**	**10-shot**	**20-shot**
Training samples	43	215	430	860
Samples/class	1	5	10	20
Epochs	20	20	10	10
Learning rate	1e-4	1e-4	5e-5	5e-5
Batch size	8	8	8	8
Optimizer	AdamW			
Trainable params	262K (chỉ classification head)			
Frozen params	151M (CLIP backbone)			

4.4.2.4 Ưu điểm

- Cải thiện performance với rất ít dữ liệu
- Training nhanh (chỉ train head, không train backbone)
- Tránh overfitting nhờ freeze pretrained weights
- Giữ lại được generalization capability của CLIP

4.4.2.5 Nhược điểm

- Performance bị giới hạn bởi frozen features
- Không thể adapt backbone cho domain-specific patterns
- Với k quá nhỏ (k=1), performance vẫn rất thấp

4.4.3 Full Fine-Tuning (Unfrozen Backbone)

4.4.3.1 Khái niệm Full fine-tuning là phương pháp huấn luyện **toàn bộ mô hình**, bao gồm cả CLIP backbone. Khác với few-shot learning, ở đây tất cả các layers đều có thể cập nhật trọng số (`requires_grad=True`).

Bảng 10: *So sánh Few-shot (Frozen) vs Full Fine-tuning (Unfrozen)*

Aspect	**Few-shot (Frozen)**	**Full Fine-tune (Unfrozen)**
Backbone status	`freeze_backbone=True`	`freeze_backbone=False`
Trainable layers	Chỉ classification head	Toàn bộ model
Trainable params	262K	546K
Training data	1-20 samples/class	100 samples/class
Learning rate	1e-4 hoặc 5e-5	2e-5 (thấp hơn)
Epochs	10-20	5 (ít hơn)
Risk of overfitting	Thấp	Cao hơn
Training time	Nhanh (85-137s)	Chậm hơn (151s)

4.4.3.2 Điểm khác biệt chính

Bảng 11: *Thông số Full Fine-Tuning*

Tham số	**Giá trị**
Training samples	3,539 (100 samples/class)
Epochs	5
Learning rate	2e-5 (thấp để stability)
Batch size	8
Optimizer	AdamW
Scheduler	Linear warmup (10% steps)
Trainable params	546K (toàn bộ model)
freeze_backbone	False

4.4.3.3 Cấu hình huấn luyện

4.4.3.4 Ưu điểm

- Tiềm năng đạt accuracy cao nhất
- Có thể adapt toàn bộ model cho domain-specific task
- Học được patterns phức tạp hơn từ dữ liệu

4.4.3.5 Nhược điểm

- Cần nhiều dữ liệu hơn (ít nhất 50-100 samples/class)
- Training chậm hơn (train toàn bộ 151M parameters)
- Nguy cơ overfitting cao nếu dữ liệu không đủ
- Có thể mất đi generalization capability của pretrained model

4.5 Thiết lập Thực nghiệm

4.5.1 Môi trường và Phần cứng

- **Platform**: Kaggle Notebook
- **GPU**: Tesla T4 x2 (16GB VRAM mỗi card)
- **Framework**: PyTorch 2.x, Transformers (HuggingFace)
- **Python**: 3.10

4.5.2 Chia tách dữ liệu

- **Test set**: 20% (885 samples) - stratified by category
- **Training set**: 80% (3,539 samples)
  - Few-shot: Lấy ngẫu nhiên k samples/class (k=1,5,10,20)
  - Full fine-tune: Sử dụng toàn bộ 100 samples/class

4.5.3 Metrics đánh giá

Theo yêu cầu của đề bài (Section 4.4), chúng tôi đánh giá tất cả các phương pháp với các metrics sau:

1. **Accuracy**: Tỷ lệ phân loại đúng tổng thể

$$\text{Accuracy} = \frac{\text{Số lượng dự đoán đúng}}{\text{Tổng số mẫu}} \tag{25}$$

2. **F1 Score (Macro)**: Trung bình cộng F1 của từng class

$$F1_{\text{macro}} = \frac{1}{C}\sum_{i=1}^{C} F1_i \tag{26}$$

3. **F1 Score (Weighted)**: Trung bình có trọng số theo số lượng samples

$$F1_{\text{weighted}} = \sum_{i=1}^{C} w_i \cdot F1_i, \quad w_i = \frac{n_i}{\sum_j n_j} \tag{27}$$

4. **Precision (Macro)**: Độ chính xác trung bình
5. **Recall (Macro)**: Độ phủ trung bình
6. **Confusion Matrix**: Ma trận nhầm lẫn 43x43
7. **Per-class Metrics**: F1, Precision, Recall cho từng category

4.6 Kết quả và Phân tích

4.6.1 So sánh Tổng quan

Bảng 12 tổng hợp kết quả của tất cả 6 thí nghiệm (1 zero-shot + 4 few-shot + 1 full fine-tuning).

Bảng 12: *So sánh tổng quan: Zero-shot vs Few-shot vs Full Fine-tuning*

Method	K	Backbone	Acc	F1 Ma	F1 We	Prec	Recall
Zero-Shot	0	Frozen	**0.758**	**0.741**	**0.740**	0.754	0.759
1-Shot	1	Frozen	0.162	0.106	0.106	0.189	0.161
5-Shot	5	Frozen	0.645	0.614	0.614	0.672	0.645
10-Shot	10	Frozen	0.399	0.367	0.367	0.487	0.398
20-Shot	20	Frozen	0.652	0.616	0.615	0.671	0.652
Full FT	100	Unfrozen	0.644	0.602	0.600	**0.653**	**0.645**

Chú thích: Acc = Accuracy, F1 Ma = F1 Macro, F1 We = F1 Weighted, Prec = Precision Macro, FT = Fine-Tuning

4.6.1.1 Quan sát chính

1. **Zero-shot đạt kết quả tốt nhất** với accuracy 75.8%, vượt trội hơn tất cả các phương pháp fine-tuning. Điều này cho thấy CLIP pretrained đã học được biểu diễn rất mạnh cho COCO dataset.
2. **1-shot performance cực kỳ thấp** (16.2%), chứng tỏ 1 sample/class không đủ để học được pattern phân loại.
3. **5-shot và 20-shot đạt kết quả tương đương nhau** (64-65%), cho thấy diminishing returns khi tăng k từ 5 lên 20.
4. **Full fine-tuning không vượt trội** (64.4%), thậm chí còn kém hơn zero-shot. Nguyên nhân có thể do:
   - Dataset size nhỏ (chỉ 43 classes với 100 samples/class)
   - Overfitting khi unfreeze toàn bộ backbone
   - CLIP pretrained đã quá mạnh cho COCO, không cần fine-tune thêm

4.6.2 Thời gian Huấn luyện và Inference

Bảng 13: *Phân tích thời gian xử lý*

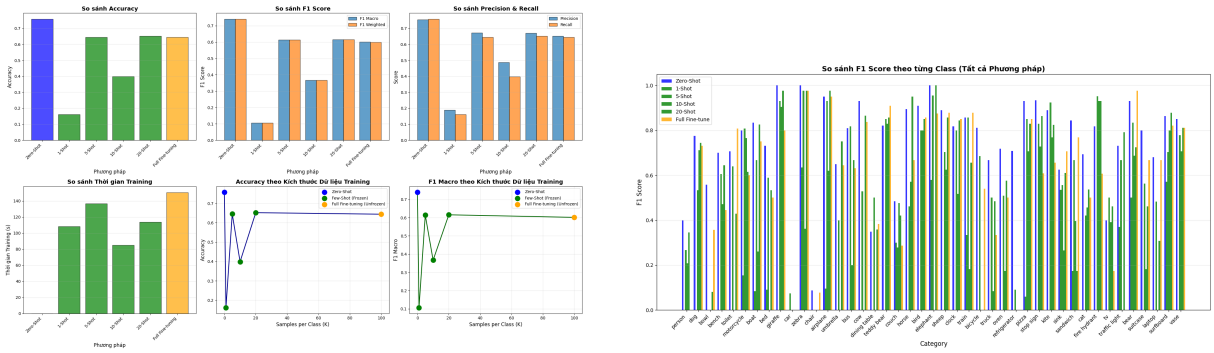
Method	Training Time (s)	Inference Time (s)
Zero-Shot	0	23.5
1-Shot	108.5	5.2
5-Shot	136.6	5.3
10-Shot	85.0	5.3
20-Shot	113.9	5.2
Full Fine-tuning	150.7	5.2

Nhận xét:

- Zero-shot có inference time cao nhất (23.5s) vì phải encode 43 text prompts cho mỗi batch.
- Các phương pháp fine-tuning có inference nhanh hơn (5s) vì chỉ cần forward pass qua classification head.
- Training time tăng dần từ few-shot (85-137s) đến full fine-tuning (151s).

4.6.3 Phân tích theo K (Samples per Class)

Hình 15 cho thấy xu hướng thay đổi của accuracy và F1 macro theo số lượng samples per class.



(a) *Accuracy theo số lượng samples per class (K)* **(b)** *F1 Macro theo số lượng samples per class (K)*

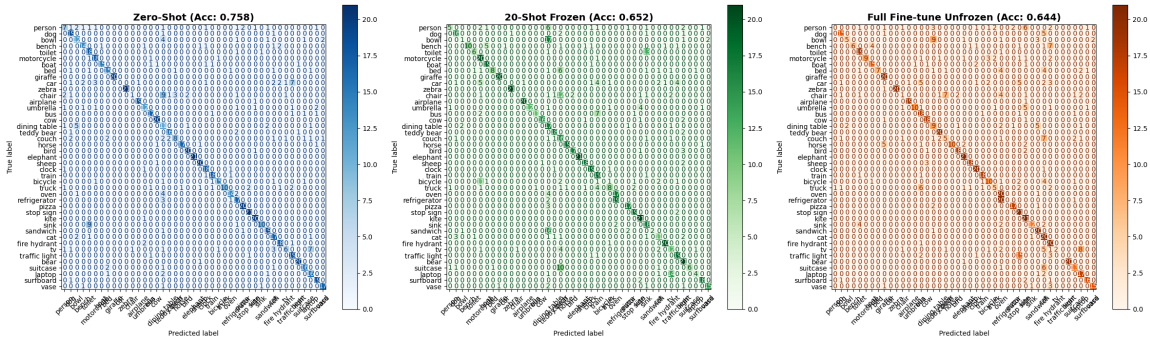
Hình 15: *Xu hướng thay đổi metrics theo K. Quan sát: performance tăng mạnh từ k=1 đến k=5, sau đó có diminishing returns.*

Quan sát:

- Accuracy tăng mạnh từ k=1 (16%) lên k=5 (64.5%)
- Từ k=5 trở đi, accuracy gần như không tăng (k=5: 64.5%, k=20: 65.2%)
- Full fine-tuning (k=100) cũng chỉ đạt 64.4%, tương đương k=5
- Điều này chứng tỏ: **k=5 đã đủ để đạt performance tốt** với frozen backbone

4.6.4 Confusion Matrices

Hình 16 trình bày confusion matrices cho ba phương pháp đại diện: Zero-shot, 5-shot (frozen), và Full fine-tuning (unfrozen).



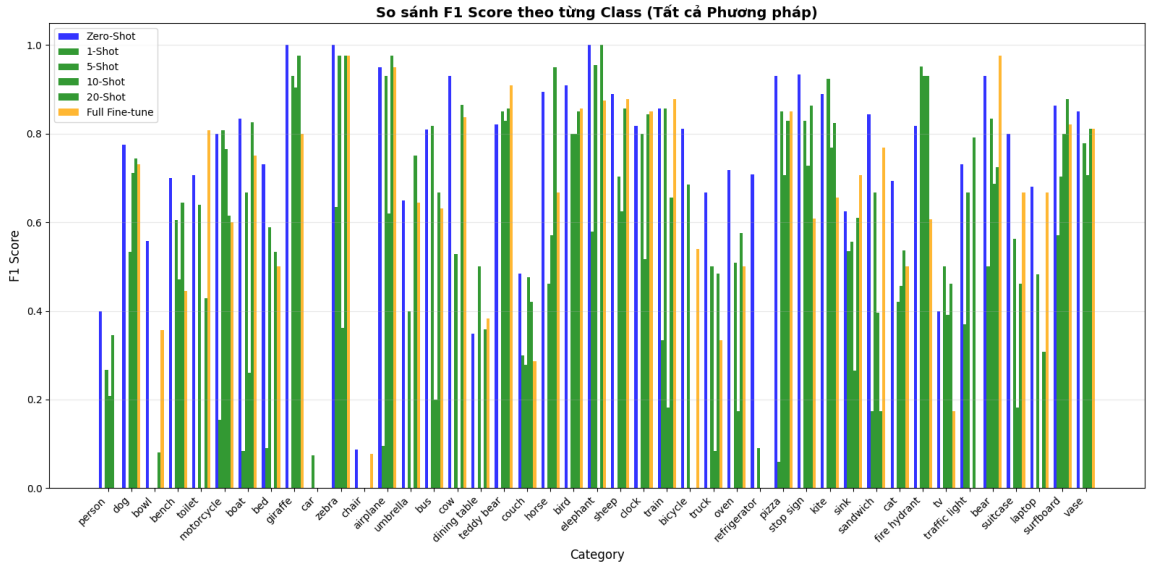
Hình 16: *Confusion matrices cho ba phương pháp (Zero-shot: 75.8%, 5-Shot: 64.5%, Full Fine-tune: 64.4%). Zero-shot có đường chéo chính rõ nhất, cho thấy ít confusion giữa các classes.*

Phân tích:

- **Zero-shot**: Đường chéo chính rất nổi bật, ít confusion giữa các classes. Cho thấy CLIP pretrained phân biệt tốt các categories COCO.
- **5-shot**: Có một số confusion tăng lên so với zero-shot, đặc biệt ở các classes tương tự nhau (ví dụ: chair vs couch, car vs truck).
- **Full fine-tuning**: Tương tự 5-shot, không cải thiện đáng kể về confusion patterns.

4.6.5 Per-class Performance

Phân tích chi tiết F1 score của từng category (Hình 17) cho thấy:



Hình 17: *So sánh F1 score theo từng category. Top performers: giraffe, zebra, elephant. Challenging: chair, person, bowl.*

4.6.5.1 Top-performing classes (F1 $>$ 0.9)

- **giraffe**, **zebra**, **elephant**: Động vật có đặc điểm ngoại hình độc nhất
- **airplane**, **train**: Phương tiện lớn, dễ phân biệt
- **stop sign**, **pizza**: Có pattern rõ ràng

4.6.5.2 Challenging classes (F1 < 0.5)

- **chair**: Bị nhầm với bench, couch do cùng là đồ nội thất
- **person**: Xuất hiện trong nhiều context khác nhau
- **bowl**, **cup**: Nhỏ, khó phân biệt trong ảnh

4.7 Kết luận

4.7.1 Tổng kết Kết quả

Qua thực nghiệm so sánh ba phương pháp phân loại đa phương thức trên COCO dataset, chúng tôi rút ra các kết luận sau:

1. **Zero-shot CLIP cực kỳ mạnh** cho COCO dataset (75.8% accuracy), vượt trội hơn cả fine-tuning. Điều này chứng tỏ CLIP pretrained đã học được biểu diễn tốt cho các object categories phổ biến.
2. **Few-shot learning hiệu quả với ít dữ liệu**. Chỉ với 5 samples/class, model đã đạt 64.5% accuracy, cải thiện đáng kể so với 1-shot (16.2%).
3. **Diminishing returns khi tăng k**: Performance gần như không thay đổi từ k=5 đến k=20, cho thấy k=5 đã đủ để học được pattern cơ bản.
4. **Full fine-tuning không luôn tốt hơn**: Với dataset nhỏ (43 classes, 100 samples/class), việc unfreeze backbone không mang lại cải thiện, thậm chí còn kém hơn zero-shot do overfitting.

4.7.2 Trade-offs giữa các Phương pháp

Bảng 14: *So sánh Trade-offs*

Tiêu chí	Zero-Shot	Few-Shot	Full Fine-tune
Training data	Không cần	Ít (5-20/class)	Nhiều (100+/class)
Training time	0s	Phút	Giờ
Accuracy	Tốt (75.8%)	Khá (64.5%)	Khá (64.4%)
Resource cost	Thấp	Trung bình	Cao
Deployment speed	Tức thì	Nhanh	Chậm
Customization	Không	Có hạn	Toàn diện
Overfitting risk	Không có	Thấp	Cao

4.7.3 Khuyến nghị Sử dụng

4.7.3.1 Khi nào dùng Zero-shot?

- Không có dữ liệu labeled
- Cần triển khai nhanh, không có thời gian training
- Task gần với CLIP pretraining (object recognition trên ảnh tự nhiên)
- Cần phân loại các classes mới liên tục

4.7.3.2 Khi nào dùng Few-shot (Frozen)?

- Có ít dữ liệu labeled (5-20 samples/class)
- Muốn cải thiện performance so với zero-shot
- Thời gian training giới hạn (vài phút)
- Muốn giữ lại generalization capability của CLIP

4.7.3.3 Khi nào dùng Full Fine-tuning (Unfrozen)?

- Có nhiều dữ liệu labeled (100+ samples/class)
- Task khác xa CLIP pretraining (medical images, satellite imagery, etc.)
- Có GPU và thời gian để train toàn bộ model
- Zero-shot và few-shot không đạt yêu cầu

4.7.4 Hướng Phát triển

Một số hướng cải thiện tiềm năng:

1. **Prompt engineering**: Thử các template prompt phức tạp hơn cho zero-shot
2. **Data augmentation**: Tăng cường dữ liệu cho few-shot learning
3. **Adapter tuning**: Thay vì full fine-tuning, chỉ thêm các adapter layers nhỏ
4. **Knowledge distillation**: Dùng zero-shot làm teacher cho few-shot student
5. **Ensemble methods**: Kết hợp predictions từ nhiều phương pháp

Tài liệu

[1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). *Learning transferable visual models from natural language supervision.* In International Conference on Machine Learning (pp. 8748-8763). PMLR.

[2] Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... & Zitnick, C. L. (2014). *Microsoft COCO: Common objects in context.* In European conference on computer vision (pp. 740-755). Springer, Cham.

[3] Snell, J., Swersky, K., & Zemel, R. (2017). *Prototypical networks for few-shot learning.* Advances in neural information processing systems, 30.